



PasswordStore Protocol Audit Report

Version 1.0

Cyfrin.io

April 15, 2026

PasswordStore Protocol Audit Report

vridhib

April 15, 2026

Prepared by: vridhib

Lead Security Researcher: vridhib

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
- High
 - [H-1] Variables of All Visibilities (Including Private) Stored On-Chain Are Public to All Users
 - [H-2] `PasswordStore::setPassword` Has No Access Controls Which Allows Non-Owners to Modify the Password
- Informational
 - [I-1] `PasswordStore::getPassword` NatSpec Incorrectly Documents A Non-Existent Parameter

Protocol Summary

PasswordStore is a protocol dedicated to storage and retrieval of a user's passwords. The protocol is designed to be used by a single user, and is not designed to be used by multiple users. Only the owner should be able to set and access this password.

Disclaimer

The auditor made all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the auditor is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash

```
1 7d55682ddc4301a7b13ae9413095feffd9924566
```

Scope

```
1 ./src/  
2 --PasswordStore.sol
```

Roles

- Owner: The user who can set the password and read the password.
- Outsiders: No one else should be able to set or read the password.

Executive Summary

The PasswordStore smart contract was audited over a one day period by one security researcher using a combination of manual review and fuzzing (Foundry). The audit focused on access controls and data privacy as defined in the protocol summary.

Overall, the contract suffers from two critical security flaws that make it unsuitable for production use without significantly redesigning the protocol's architecture. First, the private password is publicly readable from the smart contract's storage. Second, The `setPassword` function lacks any access control, allowing any user to change the password. Thus, these discovered violations of the core security assumptions (only the owner can set or read the password) render the contract's intended functionality and use case inoperable.

Additionally, one informational issue was found regarding incorrect NatSpec documentation.

Due to the severity of the findings, the contract should not be deployed as is. It is recommended to redesign the storage mechanism architecture (e.g., using off-chain encryption) and add proper access control modifiers. A follow-up audit is strongly advised after implementing the recommended changes.

Issues found

Severity	Number of Issues Found
High	2
Medium	0
Low	0
Info	1

Severity	Number of Issues Found
Total	3

Findings

High

[H-1] Variables of All Visibilities (Including Private) Stored On-Chain Are Public to All Users

Description:

All data stored on-chain is visible to anyone, and can be read directly from the blockchain. The `PasswordStore : s-password` variable is intended to be a private variable that can only be accessed through the `PasswordStore : getPassword` function (which has owner-restricted access control).

We show one such method of reading any data off-chain below.

Impact:

Anyone can read the private password, severely breaking the functionality of the protocol.

Proof of Concept:

(Proof of Code)

The below test case shows how anyone can read the password directly from the blockchain.

1. Create a locally running chain

```
1 make anvil
```

2. Deploy the contract to the chain

```
1 make deploy
```

3. Run the storage tool We use 1 because that is the storage slot of `s_password` in the contract.

```
1 cast storage 0x5FbDB2315678afecb367f032d93F642f64180aa3 1 --rpc-url  
https://127.0.0.1:8545
```

You will get an output that looks like this: 0x6d7950617373776f726400000000000000000000000000000000

You can then parse that hex value to a string with:

[illegible]

And get an output of:

```
1 myPassword
```

Recommended Mitigation:

Due to this glaring security issue, the overall architecture of the contract should be redesigned. The user could encrypt the password off-chain, and then store the encrypted password on-chain. However, this would require the user to remember another password off-chain to decrypt the password. As a result, you should additionally remove the view function as you would not want the user to accidentally send a transaction with the password that decrypts your password.

[H-2] PasswordStore::setPassword Has No Access Controls Which Allows Non-Owners to Modify the Password

Description:

The `PasswordStore::setPassword` function is set to be an `external` function that takes a `newPassword` parameter to set the new password. However, the NatSpec documentation for the function indicates that only the owner can set a new password, listing the requirement as: `This function allows only the owner to set a new password..`

```
1 function setPassword(string memory newPassword) external {
2     // @audit - There is no access control restricting non-owner
3     // users from setting the password
4     s_password = newPassword;
5     emit SetNetPassword();
6 }
```

Impact:

This lack of access control will allow anyone to set or change the password of the contract, severely breaking the intended contract functionality.

Proof of Concept:

Add the following to the `PasswordStore.t.sol` test file

Code

```
1 function test_anyone_can_set_password(address randomAddress) public {
2     vm.assume(randomAddress != owner);
3     string memory expectedPassword = "myNewPassword";
4     passwordStore.setPassword(expectedPassword);
5
6     vm.prank(owner);
7     string memory actualPassword = passwordStore.getPassword();
8     assertEq(actualPassword, expectedPassword);
9 }
```

Recommended Mitigation:

Add an access control conditional to the `setPassword` function.

```
1 if (msg.sender != s_owner) {
2     revert PasswordStore__NotOwner();
3 }
```

Informational

[I-1] PasswordStore::getPassword NatSpec Incorrectly Documents A Non-Existent Parameter

Description:

```
1 /*
2     * @notice This allows only the owner to retrieve the password.
3     * @param newPassword The new password to set.
4     */
5 function getPassword() external view returns (string memory) {
```

The `PasswordStore::getPassword` function signature is `getPassword()`, but the NatSpec documentation indicates that it should be `getPassword(string)`

Impact:

The NatSpec is incorrect.

Recommended Mitigation:

Remove the incorrect NatSpec line

```
1 - * @param newPassword The new password to set.
```